

Integración Web

Funciones más útiles de las librerías relacionadas con la integración web.

1. requests:

- **get(url, params=None, **kwargs)**: Realiza una solicitud GET a la URL especificada.
- **post(url, data=None, json=None, **kwargs)**: Realiza una solicitud POST a la URL especificada con los datos proporcionados.
- **put(url, data=None, **kwargs), delete(url, **kwargs), head(url, **kwargs), options(url, **kwargs)**: Realizan solicitudes PUT, DELETE, HEAD y OPTIONS, respectivamente.
- **status_code**: Atributo que indica el código de estado de la respuesta HTTP.
- **text**: Atributo que contiene el contenido de la respuesta HTTP como una cadena de texto.
- **json()**: Método que convierte el contenido de la respuesta HTTP en un objeto Python si el contenido es JSON.

2. BeautifulSoup:

- **BeautifulSoup(html, parser='html.parser')**: Crea un objeto BeautifulSoup a partir de una cadena de texto HTML.
- **find(name, attrs={}, recursive=True, text=None, **kwargs), find_all(name, attrs={}, recursive=True, text=None, limit=None, **kwargs)**: Encuentran el primer o todos los elementos que coinciden con los criterios especificados.
- **select(selector)**: Selecciona elementos usando selectores CSS.
- **get_text()**: Obtiene el texto dentro de un elemento BeautifulSoup.
- **parent, children, next_sibling, previous_sibling**: Atributos para navegar por la estructura del árbol DOM.

3. Selenium:

- **webdriver.Chrome(executable_path='path/to/chromedriver')**: Inicializa un navegador Chrome controlado por WebDriver.
- **get(url)**: Carga una URL en el navegador.
- **find_element_by_xpath(xpath), find_elements_by_xpath(xpath)**: Encuentran uno o todos los elementos que coinciden con el XPath especificado.

- **find_element_by_css_selector(css_selector), find_elements_by_css_selector(css_selector):** Encuentran uno o todos los elementos que coinciden con el selector CSS especificado.
- **click(), send_keys(keys):** Realizan clic en un elemento o envían teclas a un elemento de entrada.
- **close(), quit():** Cierran la ventana actual o la sesión del navegador, respectivamente.

Ejemplo:

Consulta una web y proporciona los artículos relacionados con una temática.

```
import requests
from bs4 import BeautifulSoup

# URL de la página web de noticias deportivas "Marca"
url = 'https://www.marca.com/'

# Realizamos una solicitud GET para obtener el contenido de la página
response = requests.get(url)

# Verificamos si la solicitud fue exitosa
if response.status_code == 200:
    # Parseamos el contenido HTML de la página
    soup = BeautifulSoup(response.text, 'html.parser')

    # Buscamos las noticias relacionadas con "Topuria"
    noticias = soup.find_all('a', string=lambda text: text and 'Topuria'
in text)

    # Imprimimos los titulares y enlaces de las noticias encontradas
    if noticias:
        print("Noticias relacionadas con Topuria:")
        for noticia in noticias:
            print("- Título:", noticia.text)
            print("  Enlace:", noticia['href'])
    else:
        print("No se encontraron noticias relacionadas con Topuria.")
else:
    print("Error al obtener el contenido de la página:",
response.status_code)
```

Ejemplo:

Entra en youtube, va a la pestaña de En directo y obtiene el título de los 5 primeros vídeos.

```
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.by import By
import time

# Inicializar el controlador de Selenium para Chrome
driver = webdriver.Chrome()

# Maximizar la ventana del navegador
driver.maximize_window()

# Navegar a la página de YouTube
driver.get("https://www.youtube.com/")

# Esperar unos segundos para que la página se cargue completamente
time.sleep(2)

# Encontrar y hacer clic en el botón de "Aceptar" en el mensaje de
cookies (si existe)
try:
    accept_button =
driver.find_element(By.XPATH, '//*[@id="content"]/div[2]/div[6]/div[1]/ytd
-button-renderer[2]/yt-button-shape')
    accept_button.click()
except:
    pass

# Esperar unos segundos para que la página de inicio se cargue
completamente
time.sleep(5)

# Encontrar y hacer clic en la pestaña de "En directo" para ir a los
vídeos destacados
videos_tab = driver.find_element(By.XPATH, '//*[@id="chips"]/yt-chip-
cloud-chip-renderer[3]')
videos_tab.click()

# Esperar unos segundos para que la página de vídeos se cargue
completamente
time.sleep(5)

# Obtener los títulos de los 5 primeros vídeos destacados
featured_videos = driver.find_elements(By.XPATH, '//*[@id="video-
title"]')[ :5]
for index, video in enumerate(featured_videos):
    print(f"Vídeo {index + 1}: {video.text}")

# Cerrar el navegador
driver.quit()
```

Ejemplo:

Uso de la API de OpenWeather.

```
import requests

# Configuración de la API de OpenWeatherMap
api_key = 'your-api-key'
url = 'http://api.openweathermap.org/data/2.5/weather'

# Parámetros de la solicitud
params = {
    'q': 'London,uk', # Cambia 'London,uk' a la ubicación de tu elección
    'appid': api_key,
    'units': 'metric' # Unidades métricas para la temperatura en Celsius
}

# Realizar la solicitud GET a la API de OpenWeatherMap
response = requests.get(url, params=params)

# Verificar si la solicitud fue exitosa (código de estado 200)
if response.status_code == 200:
    # Obtener los datos de la respuesta en formato JSON
    data = response.json()

    # Extraer información relevante
    ciudad = data['name']
    temperatura = data['main']['temp']
    descripcion = data['weather'][0]['description']

    # Imprimir la información del tiempo
    print(f"The weather in {ciudad} is {descripcion}. The temperature is {temperatura}°C.")
else:
    print("Error obtaining the weather:", response.status_code)
```